

Data quality checks and validation

Table constraints and pipeline expectations, the three violation actions, and where each check belongs.

A quality check is not complete until you have answered three questions: **what is valid, what happens to the invalid, and how will anyone find out.**

Two mechanisms

TABLE CONSTRAINTS

Enforced by Delta at write time

- NOT NULL and CHECK constraints on the table itself
- Invalid data is rejected; the write fails
- Works on any Delta table, no pipeline needed
- No choice of action — violation means failure

PIPELINE EXPECTATIONS

Applied during transformation

- Defined in Lakeflow declarative pipelines
- Choose to warn, drop the row, or fail the update
- Metrics recorded per expectation
- Only available inside a pipeline

THE TIEBREAKER

Constraints are a property of the table. Expectations are a property of the pipeline that fills it. Both can apply to the same data.

Table constraints

CONSTRAINT	MEANING
NOT NULL	This column may never contain a null
CHECK (amount > 0)	Every row must satisfy this boolean expression
ALTER TABLE t ADD CONSTRAINT name CHECK (...)	Add one to an existing table
ALTER TABLE t DROP CONSTRAINT name	Remove it

ON THE EXAM

Adding a CHECK constraint verifies the existing rows first. If any current row violates it, the ALTER fails.

What a constraint violation looks like

- ◆ **The write fails.** The whole transaction, not just the offending row.
- ◆ **Because Delta writes are atomic,** nothing from that write lands. There is no partial result.
- ◆ **That is strong protection and a blunt instrument.** One bad row stops a batch of ten million.
- ◆ **Use constraints for rules that must never be broken,** and expectations where you need a softer response.

ON THE EXAM

A constraint is a promise about the table that holds regardless of who writes to it. That is exactly its value and its cost.

Every expectation has three parts: a **name**, a **constraint**, and an **action**. The action is the part people forget.

The three violation actions

ACTION	SQL	WHAT HAPPENS TO THE INVALID RECORD
Warn	(no ON VIOLATION clause)	Written to the target anyway, and counted in metrics
Drop	ON VIOLATION DROP ROW	Discarded before the write. The count is logged.
Fail	ON VIOLATION FAIL UPDATE	The update stops. Partial updates are rolled back.

ON THE EXAM

Warn is the default. Retaining invalid records is what happens when you write an expectation with no ON VIOLATION clause.

When to choose each

- ◆ **Warn** when you want visibility without changing behaviour. Good for measuring a problem before acting on it.
- ◆ **Drop** when bad rows are expected and must not propagate downstream, and losing them is acceptable.
- ◆ **Fail** when any invalid record indicates a serious upstream problem that a human must look at.
- ◆ **A failure affects a single flow**, not the other flows in the pipeline.

ON THE EXAM

Fail is the right choice more often than people think, and the wrong one whenever the pipeline feeds something that must stay live.

The same expectation, two languages

SQL

```
CONSTRAINT valid_id EXPECT (id IS NOT NULL)
```

```
... EXPECT (amount >= 0) ON VIOLATION DROP ROW
```

```
... EXPECT (id IS NOT NULL) ON VIOLATION FAIL UPDATE
```

PYTHON

```
@dp.expect("valid_id", "id IS NOT NULL")
```

```
@dp.expect_or_drop("amount_ok", "amount >= 0")
```

```
@dp.expect_or_fail("id_present", "id IS NOT NULL")
```

ON THE EXAM

There are six Python decorators: the three above, plus `expect_all`, `expect_all_or_drop` and `expect_all_or_fail` for several rules at once.

A currency note on the Python syntax

- ◆ Older material shows `import dlt` and `@dlt.expect`. Your question bank almost certainly does.
- ◆ The current form is from `pyspark import pipelines as dp`, then `@dp.expect`.
- ◆ The old names still work. This is a rename, not a breaking change.
- ◆ Recognise both. The exam may use either, and the behaviour is identical.

ON THE EXAM

Same rename as module 32 collects: Delta Live Tables became Lakeflow Spark Declarative Pipelines, and the module prefix moved with it.

Naming expectations

- ◆ The name is an identifier used to track and monitor that expectation over time.
- ◆ It must be unique within a dataset.
- ◆ It can be reused across datasets in the same pipeline, which is how you track one rule in several places.
- ◆ Choose a name that says what is being validated, since it is what appears in the metrics.

ON THE EXAM

`valid_customer_age` tells you what failed. `constraint_3` tells you nothing, and it is the name you will see at three in the morning.

Where the metrics go

- ◆ The pipeline UI has a **Data quality tab** on each dataset with expectations defined.
- ◆ The event log holds the **same metrics**, and can be queried like any table.
- ◆ **Fail records no metrics**. Because the update fails on detection, there is nothing to report.
- ◆ **Fail does give a detailed error**, naming the violated expectation and showing the offending input record.

ON THE EXAM

That third point is counter-intuitive. The strictest action is the one that leaves you with the least measurement.

The quarantine pattern

- ◆ **Neither drop nor fail keeps the bad rows for inspection.** Drop discards them; fail stops everything.
- ◆ **Quarantine writes them somewhere instead:** a flag column, or a separate table.
- ◆ **Build it with a boolean column** recording whether each row passed, then split downstream on that column.
- ◆ **Use it when bad records must be investigated** rather than merely counted or discarded.

ON THE EXAM

Quarantine is the answer whenever a scenario says invalid rows must be reviewed, corrected or reprocessed later.

Three dimensions worth checking

- ◆ **Nullability.** Required values are present. NOT NULL, or an IS NOT NULL expectation.
- ◆ **Cardinality.** Values are unique where they should be. Usually a count against a count distinct.
- ◆ **Range.** Values fall within acceptable bounds. A CHECK constraint or a BETWEEN expectation.
- ◆ **Referential integrity is not enforced.** Delta has no foreign keys that reject writes; use a left_anti join to detect orphans.

ON THE EXAM

Cardinality is the one that catches the fan-out from module 11. A silver table should have one row per business key.

Where checks belong

- ◆ **Not on the way into bronze.** Bronze takes everything, as module 3 established.
- ◆ **Bronze to silver is the main place.** This is where records become trustworthy, so this is where you decide.
- ◆ **Silver to gold checks aggregate sanity:** totals, expected row counts, no unexpected nulls in dimensions.
- ◆ **Check early enough to stop bad data spreading,** and late enough that you still have the original.

ON THE EXAM

A check placed in gold has already let bad data into silver, where several other gold tables may have read it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Bad rows are still in the table despite expectations	Warn is the default. No ON VIOLATION clause was given.
The pipeline must not stop, bad rows must not propagate	ON VIOLATION DROP ROW
Any invalid record means something is seriously wrong	ON VIOLATION FAIL UPDATE
Invalid rows must be reviewed and corrected later	Quarantine, not drop
The rule must hold no matter who writes to the table	A table constraint, not an expectation
The team wants counts of how many rows failed	Warn or drop. Fail records no metrics.
Adding a CHECK constraint failed	Existing rows already violate it

ON THE EXAM

Seven lines. The first is the most commonly tested fact in the module.

A team adds an expectation to a streaming table: `CONSTRAINT valid_amount EXPECT (amount >= 0)`. They deploy it and confirm the pipeline runs successfully.

A week later, an analyst reports that the silver table still contains rows with negative amounts. The pipeline's data quality tab shows several thousand failing records.

What is happening?

- ☐ A The expectation is not being evaluated; the syntax is wrong
- ☐ B With no ON VIOLATION clause the action is warn, so invalid records are written to the target and merely counted
- ☐ C Expectations only apply to new records, and these predate the deployment
- ☐ D The data quality tab is reporting records from the source, not the target

◆ PAUSE AND DECIDE

What is happening?

A The syntax is wrong

The data quality tab is showing failing counts, which means the expectation is being evaluated correctly. It is working exactly as written.

C Only applies to new records

Would explain old rows, not the several thousand currently failing. Those are being evaluated now and written anyway.

B No ON VIOLATION clause means warn, so records are written and counted

Correct. Retaining invalid records is the documented default. The expectation measures the problem; it was never told to act on it.

D The tab reports source records

It reports records evaluated against the expectation as they flow into the target. The counts and the bad rows are the same records.

THE SIGNAL

The failing count in the UI is the evidence that everything is working. Warn means observe, not enforce.

A pipeline ingests sensor readings. About 0.5% of readings arrive with an impossible temperature, caused by a known hardware fault. Those readings must not reach the gold aggregates.

The pipeline feeds an operational dashboard that the plant team relies on continuously. It must not stop. The engineering team also wants to track how often the fault occurs, to justify replacing the sensors.

Which action fits?

- | | | | |
|----------------------------|------------------------------------|----------------------------|--|
| ☐ A | EXPECT with no ON VIOLATION clause | ☐ B | ON VIOLATION DROP ROW |
| ☐ C | ON VIOLATION FAIL UPDATE | ☐ D | A CHECK constraint on the target table |

◆ PAUSE AND DECIDE

Which action fits?

A EXPECT with no ON VIOLATION

Tracks the fault, and writes the impossible readings into the target, so they reach the gold aggregates. Fails the first requirement.

C ON VIOLATION FAIL UPDATE

Stops the update on the first bad reading, so the dashboard the plant team relies on goes down roughly every 200 readings.

B ON VIOLATION DROP ROW

Correct. Bad readings are discarded before the write, the pipeline keeps running, and the count of dropped records is logged in the metrics.

D A CHECK constraint

Rejects the whole write, which stops the pipeline just as surely as fail does, and records no per-rule metrics.

THE SIGNAL

Three requirements: exclude the rows, keep running, and count the occurrences. Drop is the only action that does all three.

A regulated silver table must never contain a row with a null `account_id`. It is currently written by one Lakeflow pipeline, which has an expectation with `ON VIOLATION FAIL UPDATE`.

The platform team is concerned that another team may later write to this table from a notebook or a job, bypassing the pipeline entirely.

What should they add?

- ☐ A Nothing; the `FAIL UPDATE` expectation already guarantees it
- ☐ B A `NOT NULL` constraint on the table, which Delta enforces against every writer
- ☐ C A second expectation with `ON VIOLATION DROP ROW` as a fallback
- ☐ D A scheduled query that checks for nulls and alerts the team

◆ PAUSE AND DECIDE

What should they add?

- A

 Nothing; FAIL UPDATE is enough
It guarantees the pipeline cannot write nulls. It says nothing about a notebook writing directly to the table, which is exactly the stated concern.
- B

 A NOT NULL constraint on the table
Correct. A constraint is a property of the table, enforced by Delta at write time regardless of which tool performs the write.
- C

 A second expectation as a fallback
Expectations only run inside the pipeline. Two of them protect against the same single path, and neither sees a notebook write.
- D

 A scheduled query that alerts
Detection after the fact. The requirement is that the table must never contain such a row, not that someone is told when it does.

THE SIGNAL

The rule must hold regardless of the writer. That phrase points at the table, not at any pipeline filling it.

A pipeline uses ON VIOLATION FAIL UPDATE on a critical expectation. It has begun failing most nights. The team wants to understand how widespread the problem is before deciding what to do, so they open the Data quality tab.

The tab shows no metrics for that expectation at all.

Why, and what should they do?

- ☐ A The metrics take 24 hours to populate; they should wait
- ☐ B Fail records no metrics, because the update fails on detection; switch temporarily to warn or drop to measure the scale
- ☐ C The expectation is misconfigured and is not running
- ☐ D Metrics are only available in the event log, never in the UI

◆ PAUSE AND DECIDE

Why, and what should they do?

A Wait 24 hours

Invents a delay. The metrics are not late; for this action they are never produced.

C It is misconfigured

It is demonstrably running — it is failing the pipeline most nights, which is precisely what it was configured to do.

B Fail records no metrics; switch temporarily to warn or drop

Correct. Because fail causes the update to fail on detection, there is nothing to aggregate. Warn measures the scale without stopping the pipeline.

D Metrics are event log only

Both surfaces exist: the Data quality tab and the event log. Neither has data here, for the same underlying reason.

THE SIGNAL

The strictest action gives the least information. If you need to size a problem, warn first and tighten afterwards.

What is valid, what happens to the invalid, and how will anyone find out. **A check that answers only the first is not a check.**